Computable Contracts by Extracting Obligation Logic Graphs

Sergio Servantez
servantez@u.northwestern.edu
Northwestern University
Evanston, Illinois, USA

Nedim Lipka
lipka@adobe.com
Adobe Research
San Jose, California, USA

Alexa Siu
asiu@adobe.com
Adobe Research
San Jose, California, USA

Milan Aggarwal
milaggar@adobe.com
Adobe Media and Data Science Research
Delhi, India

Balaji Krishnamurthy
kbalaji@adobe.com
Adobe Media and Data Science Research
Delhi, India

Aparna Garimella
garimell@adobe.com
Adobe Research
Delhi, India

Kristian Hammond
Kristian.Hammond@northwestern.edu
Northwestern University
Evanston, Illinois, USA

Rajiv Jain
rajijain@adobe.com
Adobe Research
San Jose, California, USA

ABSTRACT

The emergence of contract specific programming languages has struggled to translate into widespread adoption of computable contracts due largely to high conversion costs. In this work, we present the first system for converting natural language contracts into code through the extraction of key entities, relationships, and formulas into a graph representation called the Obligation Logic Graph (OLG). This approach allows the semantic meaning of contract obligations, including dependencies between obligations, to be captured through the OLG and mapped to code downstream. We also introduce OLG extraction as a new joint entity and relation prediction task for legal contracts, and present the Contract-OLG dataset, consisting of 1,876 contract provisions, 18,597 entities and 18,170 relationships. We perform detailed experiments to understand the capabilities of state-of-the-art Transformer and graph-based models at completing these tasks, and identify where there is currently a significant gap between human expert and machine performance, particularly for relation extraction.

CCS CONCEPTS

• **Computing methodologies** → **Natural language processing**; **Information extraction**; **Machine learning**; • **Applied computing** → **Law**.

KEYWORDS

natural language processing, information extraction, computable contracts, obligation logic graph

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICAIL 2023, June 19–23, 2023, Braga, Portugal
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0197-9/23/06…$15.00
https://doi.org/10.1145/3594536.3595162

ACM Reference Format:
Sergio Servantez, Nedim Lipka, Alexa Siu, Milan Aggarwal, Balaji Krishnamurthy, Aparna Garimella, Kristian Hammond, and Rajiv Jain. 2023. Computable Contracts by Extracting Obligation Logic Graphs. In *Nineteenth International Conference on Artificial Intelligence and Law (ICAIL 2023), June 19–23, 2023, Braga, Portugal.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3594536.3595162

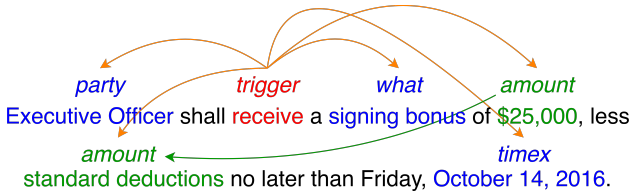

Figure 1: A simple obligation from our dataset with annotations for the OLG extraction task. Obligation trigger is red, obligation attributes are blue, and formula entities are green. Edges represent relations (orange edges are obligation argument roles and the green edge is a subtraction relation).

1 INTRODUCTION

The nascent field of computable contracts focuses on representing contracts in symbolic form, typically code, which can be easily understood by machines. Enabling machines to understand legal contracts unlocks tremendous value, including automating execution, validation and scenario planning. The promise of computable contracts has led to the emergence of domain-specific languages (DSL) - programming languages specifically designed to represent legal documents or statutes. Yet despite the existence of several of these DSLs [14, 22, 25], very few contracts or other legal documents have been converted into these code representations due to the high cost of conversion. The conversion process can be described as pair programming between a lawyer and a programmer. The lawyer understands the contract, the programmer understands the code and they must communicate to ensure the code is a faithful representation of the contract. Since this process requires a high

degree of technical and legal skills, and a significant time commitment from both, it does not scale beyond commonly used templates to the number of agreements created and revised every day.

Our work is motivated by the need to simplify and work towards automating this conversion process to assist legal professionals in moving towards computational contracts. We choose to model this problem as an information extraction task in which we extract the key entities, relationships, and formulas needed to create a computational representation of the business logic in a clause, centered around obligations in the contract. We call this graph representation the Obligation Logic Graph (OLG), and also demonstrate how the representation can be easily converted to a popular computational contract language (Accord) [25]. While recent advances in natural language processing (NLP) [6] have enabled the direct conversion between language and code, we believe this intermediate representation shown in Figure 2 is easier for legal professionals to understand and work with. Additionally, we believe enabling a deeper semantic representation of natural language contracts will benefit both lawyers and non-legal professionals in reading, understanding, and authoring agreements.

Our research also explores how extracting these computational representations can be automated using state-of-the-art NLP methods. However, a major obstacle prior to modeling, and to legal NLP tasks in general, is the scarcity of richly annotated datasets and the high cost of creating them. Unlike typical NLP datasets that can be annotated through crowdsourcing marketplaces like Mechanical Turk, annotating legal datasets requires legal expertise which narrowly restricts the pool of suitable annotators. The type of richly annotated contract data needed to perform the modeling task described above simply does not exist. And so taking on this task also required us to take on creating the necessary data, including defining a meaningful representation of entities and relations to capture the intent within the clauses. Capturing such rich annotations quickly and intuitively also required us to design a custom interface and assemble a group of legal professionals to create this dataset.

Our main research contributions are:

- We present the first system, to our knowledge, for converting natural language contracts to a code representation through the automatic extraction of obligation logic graphs and conversion to the Accord language.
- We introduce OLG extraction as new joint entity and relation extraction task, and introduce the Contract-OLG dataset, which consists of 1,876 contract provisions, 18,597 entities (7 unique categories) and 18,170 relations (24 unique categories). Our intent is to release this dataset to motivate future work into extracting rich semantic and computational representations from contract text. We also perform detailed experiments to understand the capabilities of state-of-the-art NLP algorithms at predicting obligation logic graphs.

2 RELATED WORK

2.1 Information Extraction

Recent work in information extraction has used various deep learning approaches to jointly model the four tasks of entity, relation, event and argument extraction [19, 28, 30]. While each of these four tasks are required for OLG extraction, our task is not centered on events but rather obligations. While we still detect event triggers, obligation triggers are far more prevalent in the Contract-OLG dataset (see Figure 4). In addition, we also extract other common occurrences in contracts, including dependencies (both logical and temporal) and mathematical formulas (e.g. base salary minus deductions). Similar to Jie [17], we approach formula detection as a relation extraction task where mathematical operations link variables to construct a tree representation of the formula. Unlike Jie [17], our task seeks only to extract the formula representation, not solve the expression, since contract language commonly contains variables that change based on context (e.g. standard deductions). Existing work has approached condition detection as a sentence splitting task where conditional sentences are split into antecedent and consequent using both rule-based [33] and deep learning methods [29]. Similar to formula detection, we approach condition detection as a relation extraction task allowing us to combine simple conditions using *and/or* relations to express more complex logical dependencies.

2.2 Legal NLP on Contracts

Existing work has explored a broad range of legal NLP tasks on contract language, including clause classification [16], contract summarization [20], and clause recommendation [1]. More related to our work, contract element extraction methods have explored identifying and extracting key contract elements, like clause headings and legislation references [3, 32]. The scarcity of expertly annotated datasets has been a limiting factor for legal NLP tasks, especially in the contract domain. Tuggener [27] and Hendrycks [16] have recently released large contract datasets to the public, but both are labeled at the clause or provision level. Our work builds on the provisions in Tuggener [27] to capture fine-grain entity and relation annotations.

2.3 Obligation Extraction

Prior work in obligation extraction has primarily focused on classification tasks at the sentential level [4, 13], while our approach extracts a graph representation of the obligation. Similar to an Event Logic Graph [10], our task constructs a graph of obligation nodes and conditional relations. We also go beyond the event logic graph architecture by capturing a rich set of obligation arguments along with their respective relations. There have been several works that explore components of the OLG, but our work brings these efforts together and even adds new components to present a unified graph which captures much of the semantic meaning of an obligation. Sancheti [26] presented a new taxonomy for deontic types and, similar to our work, introduced a dataset for party-specific deontic modality detection with trigger span detection. Mathur [21] presented a parser for extracting temporal dependency graphs from contracts at the document level. Funaki [12] introduced a contract corpus annotated in XML format to identify obligations, parties, conditions and exceptions. However, their XML representation captures the entire span of an obligation or condition without capturing the many obligation arguments and relations we identify in the OLG, including obligation triggers and temporal dependencies. Their dataset is also not publicly available and is more than an

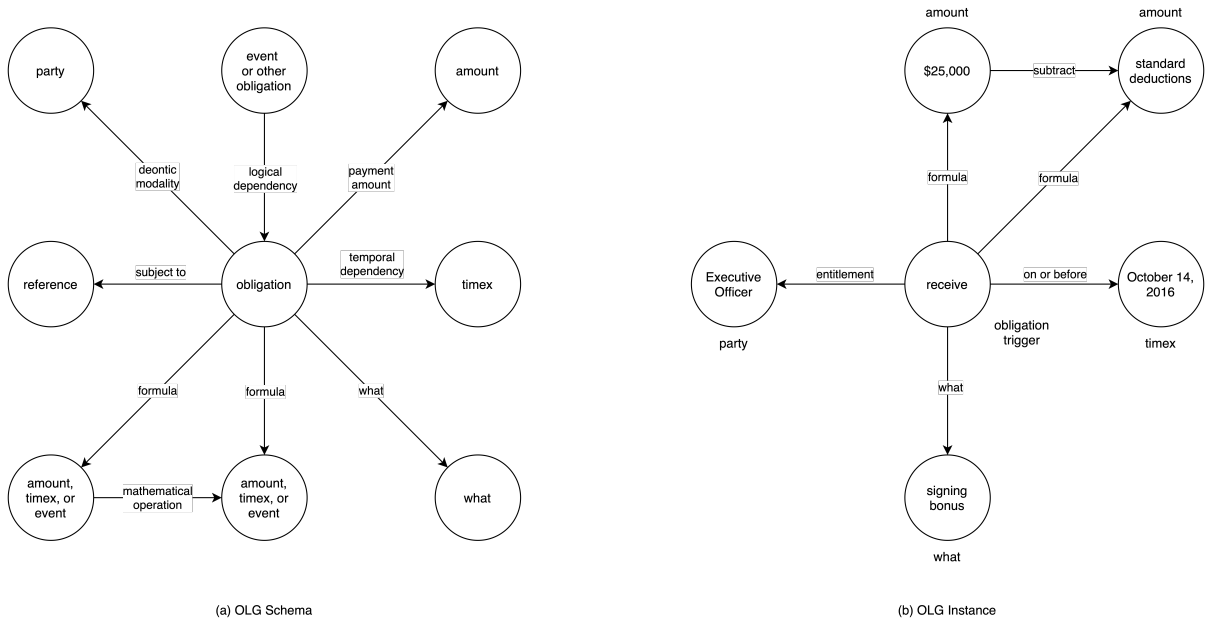

Figure 2: (a) OLG schema showing entity and relation types (b) OLG instance for obligation shown in Figure 1.

order of magnitude smaller than our dataset. Additionally, we are the first, to our knowledge, to explore complex condition extraction and formula extraction as a subtask of obligation extraction.

2.4 Computable Contracts

Several DSL for legal documents and statutes have emerged to address the need for computable contracts. A general survey [7] on computational law and contracts provides pointers to several systems as well as background on the field. LegalRuleML [23] is an XML formatted representation for legal knowledge, including rules and norms, that has been used to automate compliance checking. Catala [22] is a DSL specifically designed to translate statutory laws into an executable representation. Stipula [8] and L4 [14] are domain specific languages tailored for agreements. Stipula provides a formal specification for logic, obligations, events, entitlements and prohibitions, and also provides an example of a rental agreement converted to this specification. L4 provides similar functionality and specializes in being more human interpretable. Similarly, the Accord Project [25] has designed their own DSL for representing computable contracts, automating contract logic and generating templatized contract text (see Section 6 for a detailed description of the Accord technology stack). Our analysis of these languages found that they ranged from initial specification to well documented language guides. Though beyond a small number of examples, it was difficult to find publicly available codified clauses or contracts in large qualities. We chose to convert our OLG representation to the Accord language since it supported many of the data structures needed for our work.

3 OBLIGATION LOGIC GRAPHS

The OLG consists of a rich set of entities (nodes) and relations (edges) to capture the semantic meaning and logic in contract text and represent it computationally. In particular, we designed it to capture the business logic needed for payment clauses, though the schema could be easily extended to capture other clause types. For example, we could easily adapt the schema to represent governing law clauses by adding an entity and relation for jurisdiction. Since the OLG must be captured through human annotations, we must balance between the schema being too simple or too complex - too simple and we do not capture enough of the obligation meaning and computational semantics, too complex and we make the annotation task onerous and time consuming. Specifically, the schema attends to the who (parties), what (subject) and when (temporal dependencies), as well as logical dependencies, formulas, and deontic modalities. Below, we enumerate these graph components and discuss their importance to the OLG. Figure 2 illustrates the OLG schema.

3.1 OLG Entity Types

Each entity is expressed as a node type and a token span of the contract text. There are 7 total entity types in the Contract-OLG dataset.

- **Obligation and Event Triggers**: Our approach centers around obligations, but we also capture events since an obligation can be conditioned on an event. We define an obligation trigger as the word span that identifies a commitment or duty. Similarly, an event trigger is a word span that identifies an occurrence, typically an action taken by a party. *Entity types: obligation trigger, event trigger.*
- **Party**: A party is a participant in the contract, typically an individual or a business. *Entity type: party.*
- **Timex**: A timex is a temporal expression, typically a specified date or a relational expression (e.g. 5 days before). *Entity type: timex.*

- **What**: The *what* entities represent the subject of an obligation. In Figure 1, "signing bonus" is a *what* entity since it represents what the party is receiving. *Entity type: what.*
- **Reference**: Often an obligation inherits part of its meaning by referencing an external document or a clause from the same contract (e.g. subject to Section 5.2). *Entity type: reference.*
- **Amount**: An amount is a numerical quantity, typically related to a payment amount. *Entity type: amount.*

3.2 OLG Relation Types

Each relation is expressed as a relation type and a directed edge between two entities. There are a total of 23 relation types in the Contract-OLG dataset.

- **Deontic Modality**: Deontic modals express the type of commitment or privilege being assigned to a party (entitlement, prohibition). These relations link obligation triggers to party entities. By representing modal expressions as a relation, we are able to express party-specific deontic modalities. *Relation types: entitlement, obligation, prohibition, permission.*
- **Logical Dependencies**: Obligations are not only interconnected, but also can be conditional on other obligations and events. We express these *if-then* relationships through logical dependency relations where the antecedent (event or obligation) triggers a consequent (obligation) based on a condition that is expressed as a relation from the former to the latter. Since obligations can be conditioned on events, we also capture event triggers in the OLG. *Relation types: if true, if false, if late.*
- **Subject To**: The *subject to* relation links obligation triggers to reference entities (e.g. subject to Section 5.2). *Relation type: subject to.*
- **Temporal**: These relations express temporal relationships from obligation and event triggers to timex entities and temporal formulas. *Relation types: before, after, on, during, reoccurring.*
- **Amount**: The amount relation links obligation triggers to amount entities and amount formulas. *Relation type: amount.*
- **What**: The *what* relation links obligation triggers to *what* entities. *Relation type: what.*
- **Formula**: For each operand in an equation, there is a formula relation linking the trigger to the operand entity. *Relation type: formula.*
- **Comparators (not used)**: For completeness, we also intended to use comparison operators such as less than or equal to. However after the annotation process was complete, we found only 7 samples with these comparators and thus decided to drop these examples. *Relation types: <,<=,>,>=,==,!=.*

3.3 OLG Formulas

We treat equations made of logical dependencies or mathematical operations as equation trees in order to maintain order of operations. We use equation trees for the following relation types:

- **Operation**: The operation relations express mathematical operations between entities. *Relation types: addition, subtraction, multiplication, division, maximum, minimum.*
- **And/Or**: These relations link logical dependencies to create complex conditions in antecedents. *Relation types: and, or.*

4 THE CONTRACT-OLG DATASET

In order to support experiments for the OLG extraction task, we construct the Contract-OLG dataset. We limit our dataset to payment-related provisions to scope the task.

4.1 Preprocessing

We started with provisions from LEDGAR [27] - a multi-label corpus of contract provisions that is available to the public. From these labeled provisions, we first identified classes with a high concentration of provisions containing payment and formula related keywords (e.g. pay, deduct). The corpus was then filtered to include only provisions that contained at least one of these classes. Note that all provisions from a class, not just those containing a keyword, were included in the filtered dataset to increase language variety. To make the annotation task more manageable, we also excluded all provisions with a length greater than 750 characters. To further increase language variety, we also removed near duplicate provisions using locality-sensitive hashing (Jaccard index > 0.7). The deduplicated dataset was then split into batches (n = 100) with each batch being assigned to an annotator.

4.2 Annotation Task

The annotators were tasked to capture all information needed to construct the OLG, and as shown in Section 3 the OLG schema is highly granular. Even at the provision level, a single example can contain multiple obligations and conditions. Capturing such a detailed representation of a contract provision requires a complex annotation task where annotators must identify a broad range of entities and relations for every example. To make this task more manageable, we designed an annotation interface shown in Figure 3 with the following goals: 1) reduce time to complete task by allowing annotators to quickly select OLG components; 2) reduce task complexity by inferring information from existing annotations; and 3) enable annotators to use the interface with minimal training. Using the interface, an annotator first identifies an obligation or event by highlighting the span of the trigger. This selection populates the interface with a list of possible obligation arguments which can be quickly filled by the annotator based on which arguments exist in the provision. This approach was motivated by work in event argument mining [18] and allows us to naturally pair every argument to its corresponding obligation and thereby infer all obligation argument relations. Inspired by Blockly and Scratch [24], the interface also allows annotators to construct equation trees and conditions by dragging and dropping interlocking blocks of entities and formulas to create new relations and equations.

Since the annotation task requires the interpretation of complex contract language, we hired lawyers and law school students from Upwork to be annotators. Candidates were initially given an annotation guideline describing the task in detail, and were later given a demo of the annotation interface. Candidates were then tested by asking each to annotate the same 10 contract provisions using the interface. From the 6 candidates that took the test, we selected the top 4 who demonstrated the strongest understanding of the task.

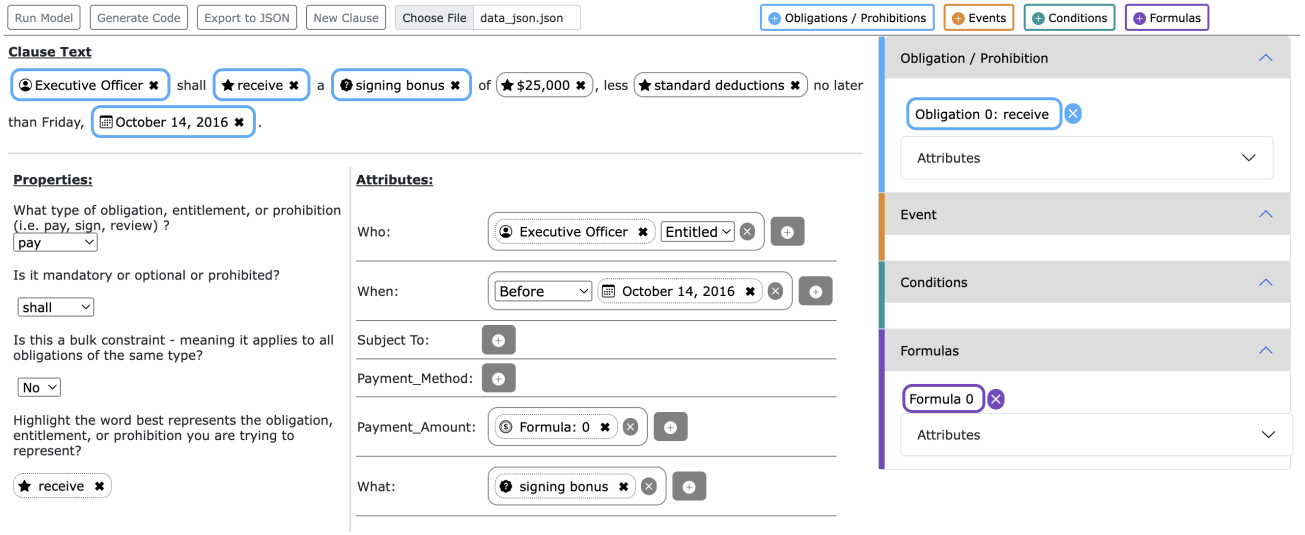

Figure 3: Screenshot of our OLG annotation interface populated with annotations for the obligation shown in Figure 1.

These annotators were then assigned batches of 100 provisions to complete at a time. Annotators were also given the option of skipping a provision if they felt like the schema would not capture its meaning or it contained no obligations. Of the data assigned, only 12% percent of provisions were skipped, providing evidence that the OLG schema could capture most of the provisions.

4.3 Dataset Statistics

There are 1,876 total provisions (see Table 2) with a mean of 1.49 (standard deviation σ = 0.68) sentences per provision and 366.18 (σ = 168.21) characters per provision. We collect 18,597 total span annotations with a mean of 9.9 (σ = 4.78) spans per provision, and 18,170 total relations with a mean of 9.69 (σ = 5.59) relations per provision. The dataset includes 30 total label categories, including 7 entity categories and 24 relation categories. Note that some related categories have been merged in Figure 5 to better visualize the distribution. For a complete list of relation categories, see Section 3. We chose to not have provisions annotated by more than one annotator, and instead focused on language variety by maximizing the number of annotated provisions. To produce inter-annotator agreement metrics, 108 provisions were randomly selected from the dataset and annotated independently again by one of the authors who is a legal expert. We chose to use these 108 provisions as the test set to minimize errors in the evaluation data.

4.4 Inter-Annotator Agreement

We measure agreement between the training and test datasets and report micro-averaged precision, recall and F1. We use a hard (exact match) and soft (partial match) metric as defined in Section 7.2. The agreement can be found in Tables 3 and 4 under the row labeled Human.

We found high agreement for entities (Trigger=87.1 and Argument=80.9 F1), though for event arguments there was a 7% gap between the hard and soft metric indicating some challenges with determining argument boundaries. The relations had lower agreement (Role=69.8 and Formula=58.7 F1) though this is partially due to the compounded errors for missed entities. This is still roughly comparable to prior work on general joint entity and relationship detection [35].

5 OLG EXTRACTION TASK

Given a contract provision, the task is to extract an OLG representation for each existing obligation. These graphs can be disjoint or, in the case of a logical dependency, be connected via a logical relation. We formulate the OLG extraction task as follows. For each obligation predict a graph $\mathbf{G}$=($\mathbf{V}$, $\mathbf{E}$) where $\mathbf{V}$ is the node set and $\mathbf{E}$ is the edge set. Each node $v_i = (t_i, s_i, e_i) \in \mathbf{V}$ represents a trigger, argument or entity where t is the node type, and s and e are the start and end token indexes, respectively. Each edge $e_{ij} = (t_{ij}, i, j) \in \mathbf{E}$ represents an argument role or relation between nodes, where t is the edge type, and i and j are node indexes for the tail and head, respectively. We further discuss the subtasks of OLG extraction below.

5.1 Entity Extraction

This task seeks to detect entity spans (start and end tokens) in the text and classify them into entity types. For example, in Figure 1 we would want to classify "Executive Officer" as a party entity and identify the contiguous span of tokens that make up that party name.

5.1.1 Obligation and Event Extraction. This task identifies obligation and event triggers (words that most clearly evoke the obligation or event), and their arguments (words that describe event or obligation attributes - e.g. parties, amounts). Triggers and arguments are subsets of entities. Returning to the example in Figure 1, more specifically "receive" is an obligation trigger, and "Executive Officer" is an obligation argument.

Table 1: Contract-OLG Dataset statistics

Dataset	Provisions	Entities	Relations	Triggers	Arguments	Roles	Formula/Logic
Train	1660	16231	15934	4178	11106	12683	3251
Dev	108	1122	1068	300	772	851	217
Test	108	1244	1168	324	852	979	189

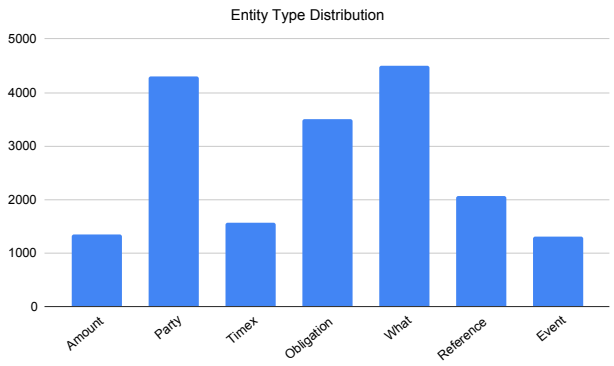

Figure 4: Distribution of entity types across the Contract-OLG dataset.

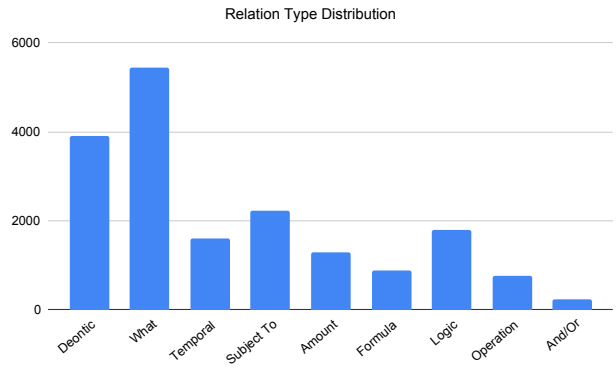

Figure 5: Distribution of relation types across the Contract-OLG dataset.

Table 2: Deontic Modality Distribution

Obligation	Entitlement	Permission	Prohibition
1712	1860	271	59

5.2 Relation Extraction

Once entities have been identified in the text, relation extraction predicts relation types between ordered pairs of entities. For example, in Figure 1 we would want to assign an entitlement relation from the "receive" entity to the "Executive Officer" entity.

5.2.1 Argument Role Extraction. Argument roles are a subset of relations. The argument role extraction task identifies relation types between triggers and arguments. The entitlement relation from Figure 1 can be more specifically defined as an argument role since it links an obligation trigger to an obligation argument.

5.2.2 Logic and Formula Extraction. Relations can also express logical dependencies between events and obligations, and mathematical operations between entities. This task identifies logical and mathematical relation types between ordered pairs of entities. For example, in Figure 1 we need to identify the subtract relation type between the fixed bonus amount ("$25,000") and "standard deductions".

6 CODE GENERATION

To connect back to our motivation for this work (enabling the conversion of natural language agreements to computable contracts) - we build a proof of concept for this idea by converting our OLG into a subset of the Accord programming language [25], which is a domain specific language built specifically for legal contracts. It captures the business logic for contracts and contains existing data structures such as clauses, obligations, parties, that make it well suited for our effort. It is made up of three parts:

- Templated Text: Accord templatizes the text, representing the key entities and elements as computational types (e.g. Strings, Numeric, Parties, DateTime) using markup.
- Data Model (Concerto): Accord uses an object oriented model to represent schema for the clause. In particular you define a set of request functions that act as an interface between external events and the clause logic, and a clause object that contains the attributes defined in the templated text.
- Logic (Ergo): Accord represents the logic for the agreement as a separate code block, where you have if/then statements or can perform computation over the variables. This is also where you emit obligations or respond to requests.

For conversion from our OLG structure to code we take inspiration from the scratch programming language [24] that converts blocks and relationships directly to code snippets. For example obligation triggers map directly to obligation emitions, event triggers to requests, obligation arguments to String (e.g. what), amounts to Double, party to Party, or timex to DateTime. The argument relationships define the object oriented structures for representing the code within the Concerto data model.

Once the Concerto data model is defined, the temporal, logical and formula relationships between obligations, events, and their arguments define any specialized logic or failure constraints that may need to be applied, and ultimately emits any obligations. For example, logical dependencies are converted to if/then statements, temporal dependencies are implemented as constraints, formulas

OLG

Executive Officer shall receive a signing bonus of $25,000, less standard deductions no later than Friday, October 14, 2016.

Entities:
party1: Executive Officer
obligation1: receive
what1: signing bonus
amount1: $25,000
amount2: standard deductions
timex1: October 14, 2016

Relationships:
subtract: amount1, amount2
entitled: obligation1, party1
before: obligation1, timex1
what: obligation1, what1
formula: obligation1, amount1
formula: obligation1, amount2
amount: obligation1, amount1
amount: obligation1, amount2

Accord Model Code

```
asset PaymentContract extends
Contract {
 --> Party party1
 o String what1
 o DateTime timex1
}

transaction clauseRequest
extends Request {
  o Double amount1
  o Double amount2
}
```

Accord Ergo Code

```
clause Obligation1Initialize(request :
clauseRequest) : Any  emits GeneralObligation {

  enforce isBefore(now(), contract.timex1 )
  else
          throw failure("Obligation1");

  emit GeneralObligation {
    entitled: [contract. party1],
    obliged: [],
    prohibited: [],
    permission: [],
    what: [contract.what1],
    subject_to: [],
    amount:[request.amount1 —
            request.amount2],
   };
  return
}
```

Figure 6: Example of code generation approach mapping the OLG extracted from Figure 1 to Accord model and logic code snippets. Highlighted fields map entities, underlines map relations.

to math equations and all obligations are emitted conditioned on these dependencies. The Accord language does not directly provide support for references so we treat them as arguments for our obligations.

To simplify the construction of these clauses we do not allow the Accord code to maintain states in the agreement and rather assume the system maintains state externally and is passed in through requests. Additionally we create an Accord clause for each obligation. Finally we do not use responses to return the clause result, but rather limit to emitting new obligations or failures. To simplify the code construction we also assume any unresolvable variables, like standard deductions, are passed to the system. Figure 6 demonstrates how our OLG representation can directly map to Accord code using our running example.

7 EXPERIMENTS

7.1 Entity and Relationship Models

We experiment on the OLG extraction task using a variety of state-of-the-art NLP models, including two extractive language models, SpERT [11] and OneIE [19], and a generative language model, GPT-3 [2]. We also break apart the OLG extraction task and experiment on entity extraction and relation extraction as separate tasks using BERT NER [9] and SAIS [34], respectively. For all models, except GPT-3, we experiment with two variants for the backbone: BERT Base and Legal BERT [5]. All models were fine-tuned on Contract-OLG using a single NVIDIA Titan V GPU.

7.1.1 SpERT [11]. A Transformer language model that has demonstrated state-of-the-art performance on joint entity and relation extraction. We fine-tune SpERT for 80 epochs with a learning rate of 5e-5 and weight decay of 1e-2.

7.1.2 OneIE [19]. A graph-based information extraction model that has demonstrated state-of-the-art performance on event, entity and relation extraction. We fine-tune OneIE for 50 epochs with a learning rate of 5e-5 and weight decay of 1e-5 for BERT parameters, and a learning rate of 1e-4 and weight decay of 1e-3 for other parameters.

7.1.3 GPT-3 [2]. Recently generative, prompt tuning approaches have shown state of the art performance for entity and relationship extraction due to pre-training. We follow the approach of DeepStruct [31] to generate entities and relationships using the prompts: "(Entity) instance of (Entity type)" and "(Entity A);(relationship);(Entity B)" sorted alphabetically. We use OpenAI for GPT-3 to build this model and train it for 4 epochs using default parameters. During inference we run with a temperature of 0 and a max generated length of 1000 tokens. This model does not output spans so it was only evaluated using the soft-metric

7.1.4 BERT NER [9]. A BERT model optimized for token classification for named entity recognition. We fine-tune BERT NER for 30 epochs with a learning rate of 1e-4 and weight decay of 1e-5 and perform inference by averaging token predictions.

	Trigger (Hard)			Trigger (Soft)			Argument (Hard)			Argument (Soft)		
Model	P	R	F1	P	R	F1	P	R	F1	P	R	F1
SpERT$_{base}$	74.6	65.4	69.7	82.0	71.9	76.6	70.1	70.1	70.1	76.6	76.6	76.6
SpERT$_{legal}$	69.6	65.7	67.6	78.4	74.1	76.2	64.9	71.7	68.1	70.5	77.9	74.0
OneIE$_{base}$	55.4	68.2	61.1	61.9	76.2	68.3	70.2	71.1	70.6	77.6	78.6	78.1
OneIE$_{legal}$	55.7	67.9	61.2	61.8	75.3	67.9	71.9	72.2	**72.0**	78.7	79.0	**78.9**
BERT NER$_{base}$	75.5	67.6	**71.3**	83.8	75.0	**79.2**	69.8	72.5	71.1	76.1	79.0	77.5
BERT NER$_{legal}$	74.2	64.8	69.2	82.7	72.2	77.1	71.1	71.4	71.3	78.7	79.0	**78.9**
GPT-3$_{curie}$	-	-	-	72.0	57.1	63.7	-	-	-	61.2	58.9	60.1
GPT-3$_{davinci}$	-	-	-	78.3	67.0	72.2	-	-	-	76.5	60.0	67.2
Human	89.3	85.0	87.1	93.9	89.4	91.6	80.8	81.0	80.9	86.9	87.2	87.1

Table 3: Precision, recall and F1 for entity extraction task on the Contract-OLG dataset. Subscript *base* is for models using BERT base, *legal* is for Legal BERT. See Section 7.2 for a description of hard and soft evaluation metrics.

7.1.5 SAIS [34]. A language model that uses evidence based data augmentation and ensemble inference to achieve state-of-the-art performance on relation extraction. We fine-tune SAIS for 30 epochs with a learning rate of 5e-5 for BERT parameters, and a learning rate of 1e-4 for other parameters.

7.2 Evaluation

We report micro-averaged precision, recall and F1 scores for all tasks, which are commonly used for entity and relationship tasks. We report both hard and soft variants of these metrics as defined below:

- `Hard Metric`: Entities must have the same type and span. Relations must have the same type and linked entities must match exactly.
- `Soft Metric`: Similar to prior work [15], credit is given for partial string overlap. Entities must have the same type, credit is given for partial overlaps if a sequence of greater than 50% of two entity strings match. Relations must have the same type and entities must pass the soft metric.

The hard metric measures whether entities and relations match exactly, while the soft metric measures whether entities and relations are semantically similar. The soft metric is meaningful because some entities, like party names, are often mentioned multiple times in a sentence. The soft metric finds agreements when annotators select the same party name, even if the spans do not match, since the selected entities are semantically equivalent. Additionally, algorithms were sometimes penalized for including adjectives or other articles, even though both predictions may have been acceptable.

7.3 Entity Extraction Performance

We report experiment results for the entity extraction task in Table 3. BERT NER (base) outperformed other models in the trigger extraction task for both hard (71.3 F1) and soft (79.2 F1) metrics. OneIE (legal) was the best performing model for argument extraction using the hard metric (72.0 F1), and tied BERT NER (legal) for the soft metric (78.9 F1). Since the GPT3 approach does not involve token spans, we cannot evaluate using the hard metric, and thus report only soft metrics for all GPT-3 results. While the GPT-3 results are generally lower than the other models, we consider these results to be promising considering the other models were specifically designed for information extraction tasks. While our baseline models show promise, we still observe considerable gaps between machine and human performance across all entity extraction tasks, indicating there is still significant room for improvement in modeling these tasks.

7.4 Relation Extraction Performance

We report experiment results for the relation extraction task in Table 4. For the argument role extraction task, SpERT (base) outperformed other models in both hard (47.1 F1) and soft (54.3 F1) metrics. OneIE was the best performing model for formula and logic extraction for both hard (34.6 F1) and soft (43.2 F1) metrics (base variant and legal variant, respectively). Compared to the entity extraction task in Table 3, the relation extraction tasks proved to be more challenging for the baseline models, especially formula and logic relations. This is partly due to error propagation - in order for a relation to be predicted correctly, both entities in the relationship must first be predicted correctly. Formulas in particular have the largest gap between human and machine performance, which could be due to there being almost 4 times fewer training points then argument roles. To better understand relation extraction as a separate task we also experiment with SAIS using oracle entities. SAIS shows limited improvement on role extraction and is outperformed by the two-stage models in formula/logic extraction indicating that this task is challenging and may require improved modeling or a larger dataset. Note that SAIS models were not considered in determining the best performing models above since the use of oracle entities gives them an advantage over other approaches.

7.5 Contract Pre-training

Unlike prior work that found pre-training in-domain to provide a few point performance boost for NER tasks [5], we did not observe significant gains on our dataset. Despite being optimized on a corpora of legal documents, the performance of Legal BERT variants on entity extraction tasks is comparable to BERT base variants, sometimes increasing by a few points and other times dropping by a similar margin. We suspect it may be due to legalBERT being an uncased model, while BERT-base was cased.

	Argument Role (Hard)			Argument Role (Soft)			Formula / Logic (Hard)			Formula / Logic (Soft)		
Model	P	R	F1	P	R	F1	P	R	F1	P	R	F1
SpERT$_{base}$	47.9	46.4	**47.1**	55.1	53.4	**54.3**	30.7	34.4	32.5	35.6	39.9	37.6
SpERT$_{legal}$	41.7	48.0	44.7	48.8	56.1	52.2	20.5	32.2	25.1	25.7	40.4	31.4
OneIE$_{base}$	45.0	40.4	42.6	52.6	47.2	49.8	35.4	33.9	**34.6**	40.6	38.8	39.7
OneIE$_{legal}$	48.3	42.1	45.0	56.4	49.2	52.6	33.7	31.1	32.4	45.0	41.5	**43.2**
GPT-3$_{curie}$	-	-	-	45.0	28.6	35.0	-	-	-	22.9	21.3	22.1
GPT-3$_{davinci}$	-	-	-	50.2	33.1	39.9	-	-	-	28.7	26.0	27.3
SAIS$^{oracle}_{base}$	57.7	53.8	55.7	60.5	56.4	58.4	31.3	35.3	33.2	32.3	36.4	34.2
SAIS$^{oracle}_{legal}$	53.3	47.3	50.1	56.0	49.7	52.7	23.5	29.0	25.9	23.9	29.5	26.4
Human	69.5	70.0	69.8	76.0	76.7	76.3	53.2	65.3	58.7	60.1	74.6	67.1

Table 4: Precision, recall and F1 for relation extraction task on the Contract-OLG dataset. Subscript *base* is for models using BERT base, *legal* is for Legal BERT. Superscript *oracle* is for models trained and evaluated using oracle entities. See Section 7.2 for a description of hard and soft evaluation metrics.

8 LIMITATIONS

While we capture a range of obligation types in this work, the OLG was designed to best represent payment related obligations. The provisions in our dataset have been specifically filtered to contain a high concentration of payment related obligations (see Section 4.1). There is no guarantee our experiment results would generalize to all contract provisions. We hope to explore a more generalized version of the OLG in future work.

9 FUTURE WORKS

We see a number of possible areas for building on this work in the future. Extending the OLG to handle additional clause types would help us understand how the tasks we explore in this work generalize to contract language in a broader context. Exploring new OLG to code generation methods would enable us to handle a broader range of logical and temporal dependency types. Inspired by the GPT-3 results, we believe exploring the effects of pre-training prompt based models on legal text could also be a rewarding path forward. To overcome the challenges we observed in the formula extraction task, it might be beneficial to explore how math word problem solving models could be adapted to handle contract formulas which include unresolvable variables and multiple disjoint formulas in a single input.

10 CONCLUSION

In this work we present the first system and task for converting natural language contracts to a computable representation through the extraction of obligation logic graphs. We show how the OLG schema defines a meaningful representation of entities and relations to capture the intent of an obligation, and present a custom interface for creating OLG annotations. To enable modeling this new OLG extraction task, we also introduce the Contract-OLG dataset consisting of 1,876 contract provisions, 18,597 entities and 18,170 relationships. Our analysis of state-of-the-art NLP models shows that there is still room for improvement to close the gap between human expert and machine performance for several OLG extraction subtasks, especially formula extraction. We intend to release the Contract-OLG dataset with the hopes this work will bring the field of computational law and the broader NLP community closer together and motivate future research.

ACKNOWLEDGMENTS

This work was supported in part by Adobe Research. The authors thank Loraine Ferrer, Nissan Daquioag, and Amritanshu Tripathi for their contributions in annotating the Contract-OLG dataset.

REFERENCES

[1] Vinay Aggarwal, Aparna Garimella, Balaji Vasan Srinivasan, Anandhavelu N, and Rajiv Jain. 2021. ClauseRec: A Clause Recommendation Framework for AI-aided Contract Authoring. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 8770–8776. https://doi.org/10.18653/v1/2021.emnlp-main.691

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. https://doi.org/10.48550/ARXIV.2005.14165

[3] Ilias Chalkidis, Ion Androutsopoulos, and Achilleas Michos. 2017. Extracting Contract Elements. In *Proceedings of the 16th Edition of the International Conference on Articial Intelligence and Law* (London, United Kingdom) *(ICAIL '17)*. Association for Computing Machinery, New York, NY, USA, 19–28. https://doi.org/10.1145/3086512.3086515

[4] Ilias Chalkidis, Ion Androutsopoulos, and Achilleas Michos. 2018. Obligation and Prohibition Extraction Using Hierarchical RNNs. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Melbourne, Australia, 254–259. https://doi.org/10.18653/v1/P18-2041

[5] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. 2020. LEGAL-BERT: The Muppets straight out of Law School. https://doi.org/10.48550/ARXIV.2010.02559

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam

McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. https://doi.org/10.48550/ARXIV.2107.03374

[7] Christopher D Clack. 2021. Languages for smart and computable contracts. *arXiv preprint arXiv:2104.03764* (2021).

[8] Silvia Crafa, Cosimo Laneve, and Giovanni Sartor. 2022. Stipula: a domain specific language for legal contracts. In *Int. Workshop Programming Languages and the Law*.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[10] Xiao Ding, Zhongyang Li, Ting Liu, and Kuo Liao. 2019. ELG: An Event Logic Graph. https://doi.org/10.48550/ARXIV.1907.08015

[11] Markus Eberts and Adrian Ulges. 2019. Span-based Joint Entity and Relation Extraction with Transformer Pre-training. *ArXiv* abs/1909.07755 (2019).

[12] Ruka Funaki, Yusuke Nagata, Kohei Suenaga, and Shinsuke Mori. 2020. A Contract Corpus for Recognizing Rights and Obligations. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*. European Language Resources Association, Marseille, France, 2045–2053. https://aclanthology.org/2020.lrec-1.251

[13] Ingo Glaser, Elena Scepankova, and Florian Matthes. 2018. Classifying semantic types of legal sentences: Portability of machine learning models. In *Legal Knowledge and Information Systems*. IOS Press, 61–70.

[14] Guido Governatori and Meng Weng Wong. 2023. Defeasible Semantics for L4. *Workshop on Programming Languages and the Law* (2023).

[15] Ralph Grishman and Beth M Sundheim. 1996. Message understanding conference-6: A brief history. In *COLING 1996 Volume 1: The 16th International Conference on Computational Linguistics*.

[16] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. https://doi.org/10.48550/ARXIV.2103.06268

[17] Zhanming Jie, Jierui Li, and Wei Lu. 2022. Learning to Reason Deductively: Math Word Problem Solving as Complex Relation Extraction. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 5944–5955. https://doi.org/10.18653/v1/2022.acl-long.410

[18] Sha Li, Heng Ji, and Jiawei Han. 2021. Document-level event argument extraction by conditional generation. *arXiv preprint arXiv:2104.05919* (2021).

[19] Ying Lin, Heng Ji, Fei Huang, and Lingfei Wu. 2020. A Joint Neural Model for Information Extraction with Global Features. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 7999–8009. https://doi.org/10.18653/v1/2020.acl-main.713

[20] Laura Manor and Junyi Jessy Li. 2019. Plain English Summarization of Contracts. In *Proceedings of the Natural Legal Language Processing Workshop 2019*. Association for Computational Linguistics, Minneapolis, Minnesota, 1–11. https://doi.org/10.18653/v1/W19-2201

[21] Puneet Mathur, Vlad Morariu, Verena Kaynig-Fittkau, Jiuxiang Gu, Franck Dernoncourt, Quan Hung Tran, Ani Nenkova, Dinesh Manocha, and Rajiv Jain. 2022. DocTime: A Document-level Temporal Dependency Graph Parser. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 993–1009.

[22] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: a programming language for the law. *Proceedings of the ACM on Programming Languages* 5, ICFP (2021), 1–29.

[23] Monica Palmirani, Guido Governatori, Antonino Rotolo, Said Tabet, Harold Boley, and Adrian Paschke. 2011. LegalRuleML: XML-Based Rules and Norms. *RuleML America* 7018 (2011), 298–312.

[24] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, et al. 2009. Scratch: programming for all. *Commun. ACM* 52, 11 (2009), 60–67.

[25] Niall Roche, Walter Hernandez, Eason Chen, Jérôme Siméon, and Dan Selman. 2021. Ergo–a programming language for Smart Legal Contracts. *arXiv preprint arXiv:2112.07064* (2021).

[26] Abhilasha Sancheti, Aparna Garimella, Balaji Vasan Srinivasan, and Rachel Rudinger. 2022. Agent-Specific Deontic Modality Detection in Legal Language. https://doi.org/10.48550/ARXIV.2211.12752

[27] Don Tuggener, Pius von Däniken, Thomas Peetz, and Mark Cieliebak. 2020. LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*. European Language Resources Association, Marseille, France, 1235–1241. https://aclanthology.org/2020.lrec-1.155

[28] Minh Van Nguyen, Viet Dac Lai, and Thien Huu Nguyen. 2021. Cross-Task Instance Representation Interactions and Label Dependencies for Joint Information Extraction with Graph Convolutional Networks. https://doi.org/10.48550/ARXIV.2103.09330

[29] Ngoc Phuoc An Vo, Irene Manotas, Octavian Popescu, Algimantas Černiauskas, and Vadim Sheinin. 2021. Recognizing and Splitting Conditional Sentences for Automation of Business Processes Management. In *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)*. INCOMA Ltd., Held Online, 1490–1497. https://aclanthology.org/2021.ranlp-1.167

[30] David Wadden, Ulme Wennberg, Yi Luan, and Hannaneh Hajishirzi. 2019. Entity, Relation, and Event Extraction with Contextualized Span Representations. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 5784–5789. https://doi.org/10.18653/v1/D19-1585

[31] Chenguang Wang, Xiao Liu, Zui Chen, Haoyun Hong, Jie Tang, and Dawn Song. 2022. DeepStruct: Pretraining of language models for structure prediction. *arXiv preprint arXiv:2205.10475* (2022).

[32] Zihan Wang, Hongye Song, Zhaochun Ren, Pengjie Ren, Zhumin Chen, Xiaozhong Liu, Hongsong Li, and Maarten de Rijke. 2021. Cross-Domain Contract Element Extraction with a Bi-directional Feedback Clause-Element Relation Network. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM. https://doi.org/10.1145/3404835.3462873

[33] Reinhardt Wenzina and Katharina Kaiser. 2013. Identifying Condition-Action Sentences Using a Heuristic-Based Information Extraction Method. In *KR4HC/ProHealth*.

[34] Yuxin Xiao, Zecheng Zhang, Yuning Mao, Carl Yang, and Jiawei Han. 2021. SAIS: Supervising and Augmenting Intermediate Steps for Document-Level Relation Extraction. https://doi.org/10.48550/ARXIV.2109.12093

[35] Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. 2019. DocRED: A large-scale document-level relation extraction dataset. *arXiv preprint arXiv:1906.06127* (2019).